



LAB MANUAL



CAMBRIDGE INSTITUTION OF TECHNOLOGY NORTH CAMPUS

Off International Airport Road, Kundana, Bengaluru - 562110

OBJECT ORIENTED PROGRAMMING WITH JAVA (1BCS302)

Prepared By

Prof. TUKARAM NAIK L C

Assistant Professor, Department of Computer Science

B.E III SEMESTER – 2026-27
(CBCS SCHEME)

Department of Computer Science and Engineering

Vision

Empowering competent, ethical Computer Science and Engineering graduates to drive innovation, entrepreneurship, and computing advancements nationally and globally.

Mission

- Mission 1: Provide quality education in Computer Science through a strong foundation in theory, practical skills, and emerging technologies.
- Mission 2: Promote innovation, entrepreneurship, ethical values, and lifelong learning to prepare graduates for professional excellence and leadership.
- Mission 3: Foster industry collaboration, research, and societal engagement to develop globally competent professionals capable of addressing real-world challenges.

Program Educational Objectives (PEOs)

PEO1:	Apply strong technical knowledge, analytical abilities, and modern computing tools to design and implement effective solutions for complex engineering problems.
PEO2:	Pursue higher education to engage in advanced research and innovation, address emerging technological challenges
PEO3:	Develop sustainable and socially relevant computing solutions with ethical values and integrity for the advancement of society

Program Specific Outcomes (PSOs)

PSO1:	Apply the knowledge of programming languages, data structures, operating systems, computer networks, and software development principles.
PSO2:	Leverage modern computing tools, algorithms, and databases to analyze complex computing problems to develop innovative software solutions.
PSO3:	Develop and implement efficient, secure and intelligent software applications by applying appropriate data, machine learning techniques, and emerging computing technologies.

PROGRAM OUTCOMES (POs)

PO's	DESCRIPTION
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

OBJECT ORIENTED PROGRAMMING WITH JAVA

(Effective from the academic year 2026 -2027) SEMESTER –III

Course Code	1BCS302	CIE Marks	50
Number of Contact Hours/Week	0:0:2:0	SEE Marks	50
Scheme: 2025		Credits -04	

COURSE OBJECTIVE

1. Introduce the fundamental concepts of Java programming, including variables, data types, operators, branching, and looping structures.
2. Develop programming skills using object-oriented programming concepts such as classes, objects, methods, and encapsulation.
3. Familiarize students with inheritance, packages, interfaces, and other mechanisms for developing modular and reusable Java applications.
4. Enable students to handle runtime errors and perform concurrent programming using exception handling and multithreading.
5. Provide an understanding of advanced Java features such as autoboxing, enumerations, and generics for developing

flexible and type-safe programs.

6. Familiarize students with suitable Integrated Development Environment (IDE) tools for designing, developing, testing, debugging, and executing Java programs.

COURSE OUTCOMES

- 1. Demonstrate proficiency in writing simple programs involving branching and looping structures.**
- 2. Construct object-oriented solutions using classes, objects, and methods.**
- 3. Develop programs using inheritance, packages, and interfaces.**
- 4. Make use of exception handling and multithreading in solving complex problems.**
- 5. Apply concepts of autoboxing, enumerations, and generics in program development.**
- 6. Choose a suitable IDE tool to design, develop, and debug Java programs.**

Program 01 : Matrix Addition

**Develop a JAVA program to add TWO matrices of suitable order N
(The value of N should be read from command line arguments).**

```
public class MatrixAddition {

    public static void main(String[] args) {

        // Check if the number of command line arguments is correct

        if (args.length != 1) {
            System.out.println("Usage: java MatrixAddition <order_N>");
            return;
        }

        // Parse the command line argument to get the order N

        int N = Integer.parseInt(args[0]);

        // Check if N is a positive integer

        if (N <= 0) {

            System.out.println("Please provide a valid positive integer for the order N.");

            return;
        }

        // Create two matrices of order N

        int[][] matrix1 = new int[N][N];

        int[][] matrix2 = new int[N][N];

        // Fill the matrices with some sample values (you can modify this as needed)

        fillMatrix(matrix1, 1);

        fillMatrix(matrix2, 2);

        // Print the matrices

        System.out.println("Matrix 1:");
        printMatrix(matrix1);
```

```
        System.out.println("\nMatrix 2:");
        printMatrix(matrix2);
        // Add the matrices

        int[][] resultMatrix = addMatrices(matrix1, matrix2);

        // Print the result matrix

        System.out.println("\nResultant Matrix (Matrix1 + Matrix2):");

        printMatrix(resultMatrix);
    }

    // Helper method to fill a matrix with sequential values
    private static void fillMatrix(int[][] matrix, int startValue) {
        int value = startValue;

        for (int i = 0; i < matrix.length; i++) {

            for (int j = 0; j < matrix[i].length; j++) {

                matrix[i][j] = value++;
            }

        }

    }

    // Helper method to add two matrices

    private static int[][] addMatrices(int[][] matrix1, int[][] matrix2) {

        int N = matrix1.length;

        int[][] resultMatrix = new int[N][N];
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {

                resultMatrix[i][j] = matrix1[i][j] + matrix2[i][j];

            }

        }

    }
```

```
        return resultMatrix;

    }

    // Helper method to print a matrix

    private static void printMatrix(int[][] matrix) {
        for (int[] row : matrix) {
            for (int value : row) {
                System.out.print(value + " ");
            }

            System.out.println();
        }
    }
}
```

Output

```
$ java MatrixAddition 3
```

Matrix 1:

1	2	3
4	5	6
7	8	9

Matrix 2:

2	3	4
5	6	7
8	9	10

Resultant Matrix (Matrix1 + Matrix2):

3	5	7
9	11	13
15	17	19

PROGRAM:02

2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations.

```
import java.util.Scanner;

public class Stack {
    private static final int MAX_SIZE = 10;
    private int[] stackArray;
    private int top;
    public Stack() {

        stackArray = new int[MAX_SIZE];

        top = -1;
    }

    public void push(int value)
    {
        if (top < MAX_SIZE - 1) {
            stackArray[++top] = value;
            System.out.println("Pushed: " + value);
        } else {
            System.out.println("Stack Overflow! Cannot push " + value + ".");
        }
    }

    public int pop() {
        if (top >= 0) {
            int poppedValue = stackArray[top--];

            System.out.println("Popped: " + poppedValue);

            return poppedValue;
        }
    }
}
```

```
    }

    Else

    {

        System.out.println("Stack Underflow! Cannot pop from an empty stack.");

        return -1; // Return a default value for simplicity
    }

}

public int peek() {
    if (top >= 0) {

        System.out.println("Peeked: " + stackArray[top]);

        return stackArray[top];

    } else {

        System.out.println("Stack is empty. Cannot peek.");

        return -1; // Return a default value for simplicity
    }

}

public void display() {
    if (top >= 0) {
        System.out.print("Stack Contents: ");
        for (int i = 0; i <= top; i++) {
            System.out.print(stackArray[i] + " ");
        }

        System.out.println();
    } else {

        System.out.println("Stack is empty.");

    }

}
```

```
}

public boolean isEmpty() {

return top == -1;
}

public boolean isFull() {

return top == MAX_SIZE - 1;
}

public static void main(String[] args) {

Stack stack = new Stack();
Scanner scanner = new Scanner(System.in);
int choice;

do {

    System.out.println("\nStack Menu:");
    System.out.println("1. Push");
    System.out.println("2. Pop");
    System.out.println("3. Peek");
    System.out.println("4. Display Stack Contents");

    System.out.println("5. Check if the stack is empty");
    System.out.println("6. Check if the stack is full");
    System.out.println("0. Exit");

    System.out.print("Enter your choice: ");

    choice = scanner.nextInt();

    switch (choice) {

        case 1:

            System.out.print("Enter the value to push: ");

            int valueToPush = scanner.nextInt();
            stack.push(valueToPush);
```

```
        break;

        case
2:
        stack.pop();
        break;

case 3:

        stack.peek();

        break;
case 4:

        stack.display();

        break;
case 5:

        System.out.println("Is the stack empty? " + stack.isEmpty());

        break;
case 6:

        System.out.println("Is the stack full? " + stack.isFull());

        break;
case 0:

        System.out.println("Exiting the program. Goodbye!");
        break;
default:

        System.out.println("Invalid choice. Please try again.");

    }

    } while (choice != 0);

    scanner.close();
}

}
```

Output

```
$ java Stack
```

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 4

Stack is empty.

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 5

Is the stack empty? true

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 6

Is the stack full? false

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 1

Enter the value to push: 10

Pushed: 10

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 1

Enter the value to push: 20

Pushed: 20

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 4

Stack Contents: 10 20

Stack Menu:

1. Push
2. Pop

3. Peek

4. Display Stack Contents

5. Check if the stack is empty

Check if the stack is full

0. Exit

Enter your choice: 3

Peeked: 20

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display Stack Contents

5. Check if the stack is empty

6. Check if the stack is full

0. Exit

Enter your choice: 1

Enter the value to push: 30

Pushed: 30

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display Stack Contents

5. Check if the stack is empty

6. Check if the stack is full

0. Exit

Enter your choice: 4

Stack Contents: 10 20 30

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display Stack Contents

5. Check if the stack is empty

6. Check if the stack is full

0. Exit

Enter your choice: 2

Popped: 30

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display Stack Contents

5. Check if the stack is empty

6. Check if the stack is full

0. Exit

Enter your choice: 3

Peeked: 20

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 4

Stack Contents: 10 20

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display Stack Contents
5. Check if the stack is empty
6. Check if the stack is full
0. Exit

Enter your choice: 0

Exiting the program. Goodbye!

PROGRAM: 03

3. To create a Java class Student with USN, Name, Branch, and Phone as data members, create student objects ,and display their details with suitable headings.

```
import java.util.Scanner;

class Student {
    String USN;
    String Name;
    String Branch;
    String Phone;

    // Constructor
    Student(String USN, String Name, String Branch, String Phone) {
        this.USN = USN;
        this.Name = Name;
        this.Branch = Branch;
        this.Phone = Phone;
    }

    // Method to display student details
    void display() {
        System.out.printf("%-15s %-20s %-15s %-15s\n",
            USN, Name, Branch, Phone);
    }
}

public class StudentDetails {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of students: ");
```

```
int n = sc.nextInt();
sc.nextLine();

Student[] students = new Student[n];

// Read student details
for (int i = 0; i < n; i++) {
    System.out.println("\nEnter details of Student " + (i + 1));

    System.out.print("USN   : ");
    String usn = sc.nextLine();

    System.out.print("Name   : ");
    String name = sc.nextLine();

    System.out.print("Branch : ");
    String branch = sc.nextLine();

    System.out.print("Phone  : ");
    String phone = sc.nextLine();

    students[i] = new Student(usn, name, branch, phone);
}

// Display student details
System.out.println("\n-----");
System.out.println("          STUDENT DETAILS");
System.out.println("-----");
System.out.printf("%-15s %-20s %-15s %-15s\n",
    "USN", "Name", "Branch", "Phone");
System.out.println("-----");

for (int i = 0; i < n; i++) {
    students[i].display();
}
```

```
System.out.println("-----");

sc.close();
}
}
```

OUTPUT:

Enter the number of students: 3

Enter details of Student 1

USN : 1AJ24CS001

Name : Rahul

Branch : CSE

Phone : 9876543210

Enter details of Student 2

USN : 1AJ24CS002

Name : Priya

Branch : CSE

Phone : 9876543211

Enter details of Student 3

USN : 1AJ24CS003

Name : Kiran

Branch : CSE

Phone : 9876543212

STUDENT DETAILS

USN	Name	Branch	Phone
1AJ24CS001	Rahul	CSE	9876543210
1AJ24CS002	Priya	CSE	9876543211
1AJ24CS003	Kiran	CSE	9876543212

Program 04 : Inheritance & Polymorphism – Shape Class

Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.

```
class Shape {  
  
    protected String name;  
    public Shape(String name) {  
  
        this.name = name;  
  
    }  
  
    public void draw() {  
        System.out.println("Drawing a " + name);  
    }  
  
    public void erase() {  
        System.out.println("Erasing a " + name);  
  
    }  
  
}  
  
class Circle extends Shape {
```

```
private double radius;

public Circle(String name, double radius) {
    super(name);
    this.radius = radius;
}

@Override

public void draw() {

    System.out.println("Drawing a circle with radius " + radius);
}

@Override

public void erase() {

    System.out.println("Erasing a circle with radius " + radius);
}
}

class Triangle extends Shape {
    private double base;

    private double height;

    public Triangle(String name, double base, double height) {
        super(name);
        this.base = base;
        this.height = height;
    }

    @Override

    public void draw() {

        System.out.println("Drawing a triangle with base " + base + " and height " + height);
    }

    @Override
```

```
        public void erase() {

            System.out.println("Erasing a triangle with base " + base + " and height " + height);

        }

    }

class Square extends Shape {
    private double side;
    public Square(String name, double side) {
        super(name);

        this.side = side;
    }

    @Override

    public void draw() {

        System.out.println("Drawing a square with side length " + side);

    }

    @Override

    public void erase() {

        System.out.println("Erasing a square with side length " + side);

    }

}

public class ShapeDemo {

    public static void main(String[] args) {
        Shape[] shapes = new Shape[3];
        shapes[0] = new Circle("Circle", 5.0);
        shapes[1] = new Triangle("Triangle", 4.0, 6.0);
        shapes[2] = new Square("Square", 3.0);
        for (Shape shape : shapes) {
```

```
        shape.draw();
        shape.erase();
        System.out.println();
    }
}
}
```

Output

```
$ java ShapeDemo
```

Drawing a circle with radius 5.0

Erasing a circle with radius 5.0

Drawing a triangle with base 4.0 and height 6.0

Erasing a triangle with base 4.0 and height 6.0

Drawing a square with side length 3.0

Erasing a square with side length 3.0

Program 05 : Resizable interface

Develop a JAVA program to create an interface Resizable with methods `resizeWidth(int width)` and `resizeHeight(int height)` that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods.

```
//Resizable interface
interface Resizable {

    voidresizeWidth(intwidth);

    voidresize Height(intheight);
}

// Rectangle class implementing Resizable interface
class Rectangle implements Resizable {

    private int width;
```

```
private int height;

public Rectangle(int width, int height) {
    this.width = width;
    this.height = height;
}

// Implementation of Resizable interface

@Override
public void resizeWidth(int width) {
    this.width = width;

    System.out.println("Resized width to: " + width);
}

@Override
public void resizeHeight(int height) {
    this.height = height;

    System.out.println("Resized height to: " + height);
}

// Additional methods for Rectangle class
public int getWidth() {
    return width;
}

public int getHeight() {
    return height;
}

public void displayInfo() {
    System.out.println("Rectangle: Width = " + width + ", Height = " + height);
}
}

// Main class to test the implementation
public class ResizeDemo {

    public static void main(String[] args) {
        // Creating a Rectangle object
        Rectangle rectangle = new Rectangle(10, 5);
    }
}
```

```
// Displaying the original information
System.out.println("Original Rectangle Info:");
rectangle.displayInfo();

// Resizing the rectangle
rectangle.resizeWidth(15);
rectangle.resizeHeight(8);

// Displaying the updated information
System.out.println("\nUpdated Rectangle Info:");
rectangle.displayInfo();
}
}
```

Output

```
$ java ResizeDemo
Original Rectangle Info:

Rectangle: Width = 10, Height = 5
Resized width to: 15

Resized height to: 8

Updated Rectangle Info:
Rectangle: Width = 15, Height = 8
```

Program 06 : Packages

Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.

Package mypack

```
// Inside a folder named 'mypack'
package mypack;

public class MyPackageClass {
    public void displayMessage() {

        System.out.println("Hello from MyPackageClass in mypack package!");
    }
}
```

```
// New utility method
public static int addNumbers(int a, int b) {
    return a + b;
}
}
```

Now, let's create the main program in a different file outside the **mypack** folder:

Package Demo class using mypack Package

```
// Main program outside the mypack folder
import mypack.MyPackageClass;

//import mypack.*;

public class PackageDemo {
    public static void main(String[] args) {
        // Creating an instance of MyPackageClass from the mypack package
        MyPackageClass myPackageObject = new MyPackageClass();

        // Calling the displayMessage method from MyPackageClass
        myPackageObject.displayMessage();

        // Using the utility method addNumbers from MyPackageClass
        int result = MyPackageClass.addNumbers(5, 3);
        System.out.println("Result of adding numbers: " + result);
    }
}
```

Organize your directory structure as follows:

```
project-directory/  
├── mypack/  
│   └── MyPackageClass.java  
└── PackageDemo.java
```

Compile the files:

```
javac mypack/MyPackageClass.java  
javac PackageDemo.java
```

Output:

```
$ java PackageDemo  
Hello from MyPackageClass in mypack package!  
Result of adding numbers: 8
```

Program 07 : Custom Exception

Develop a JAVA program to raise a custom exception (user defined exception) for DivisionByZero using try, catch, throw and finally.

```
// Custom exception class  
class DivisionByZeroException extends Exception {  
    public DivisionByZeroException(String message) {  
        super(message);  
    }  
}  
  
public class CustomExceptionDemo {  
    // Method to perform division and throw custom exception if denominator is zero  
    static double divide(int numerator, int denominator)  
        throws DivisionByZeroException
```

```
{  
    if (denominator == 0) {  
        throw new DivisionByZeroException("Cannot divide by zero!");  
    }  
    return (double) numerator / denominator;  
}  
  
public static void main(String[] args) {  
    int numerator = 10;  
    int denominator = 0;  
  
    try {  
        double result = divide(numerator, denominator);  
        System.out.println("Result of division: " + result);  
    } catch (DivisionByZeroException e) {  
        System.out.println("Exception caught: " + e.getMessage());  
    } finally {  
        System.out.println("Finally block executed");  
    }  
}
```

Output

\$ java CustomExceptionDemo

Exception caught: Cannot divide by zero!

Finally block executed.

Program 08: Thread Class

Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

```
class MyThread extends Thread {  
    // Constructor calling base class constructor using super  
    public MyThread(String name) {  
        super(name);  
        start(); // Start the thread in the constructor  
    }  
  
    // The run method that will be executed when the thread starts  
    @Override  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(Thread.currentThread().getName() + " Count: " + i);  
            try {  
                Thread.sleep(500); // Sleep for 500 milliseconds  
            } catch (InterruptedException e) {  
                System.out.println(Thread.currentThread().getName() + " Thread interrupted.");  
            }  
        }  
    }  
}  
  
public class ThreadConcurrentExample {  
    public static void main(String[] args) {  
        // Create an instance of MyThread
```

```
MyThread myThread = new MyThread("Child Thread");

// Main thread
for (int i = 1; i <= 5; i++) {
    System.out.println(Thread.currentThread().getName() + " Thread Count: " + i);
    try {

        Thread.sleep(500); // Sleep for 500 milliseconds
    }

    catch (InterruptedException e) {
        System.out.println(Thread.currentThread().getName() + " Thread interrupted.");
    }
}
}
```

Output

```
$ java ThreadConcurrentExample
```

```
main Thread Count: 1
```

```
Child Thread Count: 1
```

```
main Thread Count: 2
```

```
Child Thread Count: 2
```

```
main Thread Count: 3
```

```
Child Thread Count: 3
```

```
main Thread Count: 4
```

```
Child Thread Count: 4
```

```
main Thread Count: 5
```

```
Child Thread Count: 5
```

Program 09: Enumeration Method

Develop a Java program using an enumeration called Department containing CSE, ISE, ECE, and MECH. Accept a department name as input and display the selected department using valueOf().

```
import java.util.Scanner;

public class DepartmentDemo {

    enum Department {

        CSE, ISE, ECE, MECH

    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter department name (CSE/ISE/ECE/MECH): ");

        String input = sc.next().toUpperCase();

        try {

            Department dept = Department.valueOf(input);

            System.out.println("Selected Department: " + dept);

        } catch (IllegalArgumentException e) {

            System.out.println("Invalid department name entered.");

        }

        sc.close();

    }

}
```

output:

Enter department name (CSE/ISE/ECE/MECH): ise

Selected Department: ISE

Program 10: Autoboxing/Unboxing

Develop a Java program to read the marks of three subjects as strings, convert them into wrapper objects, calculate the total and average using autoboxing/unboxing and display the result.

```
import java.util.Scanner;

public class MarksWrapperDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter marks of subject 1: ");

        String s1 = sc.next();

        System.out.print("Enter marks of subject 2: ");

        String s2 = sc.next();

        System.out.print("Enter marks of subject 3: ");

        String s3 = sc.next();

        // Convert String to Integer wrapper objects

        Integer marks1 = Integer.valueOf(s1);

        Integer marks2 = Integer.valueOf(s2);

        Integer marks3 = Integer.valueOf(s3);

        // Autoboxing/unboxing during arithmetic

        int total = marks1 + marks2 + marks3;
```

```
double average = total / 3.0;
```

```
System.out.println("Total Marks = " + total);
```

```
System.out.println("Average Marks = " + average);
```

```
sc.close();
```

```
}
```

```
}
```

output

Enter marks of subject 1: 78

Enter marks of subject 2: 85

Enter marks of subject 3: 92

Total Marks = 255

Average Marks = 85.0

